PFTDC

Problem Framing & Training Data Construction

EVA Machine Learning Reasoning Layer — Module Specification

Version 1.0 | References: EVA-ML-Master.docx

1. Purpose

The Problem Framing & Training Data Construction module determines whether machine learning is appropriate for the current analysis session and, if so, defines the exact learning problem and constructs the dataset the model will learn from. It converts analytical intent into a formal learning task.

Why This Module Is Critical
Machine learning models do not fail primarily due to algorithms. They fail because they are trained to solve the wrong problem. The PFTDC module prevents incorrect modeling by ensuring the prediction task is meaningful, well-defined, and grounded in the user's objective.

> *The system prefers no model over a misleading model.*

2. Design Philosophy

A dataset does not automatically imply a prediction task. Before any model is created, EVA must answer three questions: What future or unknown outcome are we predicting? Does predicting it provide value to the user? Is the dataset capable of supporting such prediction? If these cannot be answered confidently, the ML pipeline must not proceed.

3. Inputs from Global Analysis Ledger

The module reads the following keys from the GAL:

- dataset_identity — row meaning, domain, time behavior classification
- user_intent_record — objective, decision type, stakeholder type
- restricted_columns — leakage risks and excluded identifiers
- hypotheses — key findings from the Investigation & Hypothesis Engine
- feature_registry — validated engineered features

4. Determining Whether Modeling Is Needed

4.1 Conditions for Acceptance

- The user needs a future or unknown outcome
- The outcome is not already known at record creation time
- The dataset contains meaningful predictors

4.2 Conditions for Rejection

- The analysis goal is purely explanatory
- No valid target variable exists
- The dataset is too small or too incomplete to support reliable learning

- Prediction would not influence any real decision

If rejected, the module records the reason in the GAL session_log and the ML pipeline ends. EVA falls back to descriptive analysis and notifies the user.

5. Problem Type Identification

If modeling is justified, the module classifies the learning problem. The classification and its justification are recorded in the model_definition_record.

Problem Type	Description
Binary Classification	Predict a yes/no outcome
Multi-Class Classification	Predict one category among many
Regression	Predict a continuous value
Time Forecasting	Predict a future value over time
Anomaly Detection	Detect unusual or unseen behavior
Clustering (Fallback)	Group entities when no explicit outcome exists but decision value remains. This is distinct from descriptive segmentation, which is not an ML task.

6. Target Variable Selection

The module confirms the target variable using Dataset Profiler candidates, user intent, and real-world interpretation.

- The target must represent a future or unknown outcome
- The target must not contain information unavailable at prediction time
- The target must be decision-relevant
- If multiple targets exist, the module prioritizes based on user objective

7. Learning View Construction

The model will not train on the raw dataset. The module constructs a Learning View — a structured dataset designed specifically for learning.

7.1 Included

- Repaired dataset rows
- Validated engineered features from the feature_registry
- Permitted columns only

7.2 Excluded

- Identifier columns
- Leakage columns and post-outcome information
- Any column in the restricted_columns GAL key

> ⚠ **Critical Rule**
> The Learning View — not the raw dataset — is the only permitted input to the training pipeline. This rule applies to all downstream modules and cannot be overridden.

8. Feature Eligibility Filtering

Every feature is evaluated before inclusion. A feature is excluded if it directly encodes the outcome, depends on future information, violates real-world availability at prediction time, or contradicts identified misinterpretation risks. Both accepted and rejected features are recorded in the model_definition_record.

9. Time-Dependent Data

If the dataset is temporal, the module enforces chronological learning rules. Training data must come strictly from earlier time periods than validation data. Random mixing of time periods is forbidden because it creates unrealistic performance estimates. The time boundary used is recorded in the model_definition_record.

10. Data Splitting Strategy

The Learning View is divided into a training set, validation set, and holdout test set. The split must preserve class distribution when relevant, respect time ordering when applicable, and avoid entity duplication across sets. Split reasoning is recorded to ensure reproducibility.

10.1 Class Imbalance

If the target variable has severe class imbalance, the module must flag this in the model_definition_record. This flag is passed to the MCG to adjust metric selection and to the TEM to adjust evaluation criteria.

11. Minimum Viability Check

Before allowing training, the module verifies sufficient record count, sufficient variation in the target, a usable feature count, and no dominant missingness. If the dataset cannot support reliable learning, modeling is halted and the reason is logged.

12. Output: model_definition_record

Field	Contents
Modeling Decision	Whether ML is used and why
Problem Type	Classification, regression, forecasting, etc.
Selected Target	Outcome variable and justification
Learning View Schema	Features allowed and their types
Excluded Features	Leakage and restricted columns with reasons
Split Strategy	How training/validation/test were created
Class Imbalance Flag	Whether severe imbalance was detected
Feasibility Assessment	Confidence in training viability

13. Operational Rules

1. No model may train without this module's approval.
2. Raw dataset must never be directly used for training.
3. All excluded columns must be recorded with justification.
4. Time leakage must be prevented through chronological splits.
5. The modeling problem must match user intent.
6. If rejected, EVA must notify the user and fall back to descriptive analysis.

14. Relationship to Other Modules

Module / Document	Relationship
Feature Intelligence Engine	Provides candidate features via feature_registry
Data Repair & Integrity Layer	Provides the clean repaired dataset
Investigation & Hypothesis Engine	Provides real-world context via hypotheses
MCG	Consumes model_definition_record
Master PRD (EVA-ML-Master.docx)	Defines global rules this module must comply with